

classicmodels_task6

May 3, 2026

1 W4111 Task 6 — Notebook (README: test Tasks 1–5)

This notebook matches `README.md`:

- **Task 6:** Cells test `MySQLDataService`, `Resources`, and the **HTTP API** (requests). Run **top to bottom** with the API already running: `python -m app.main` from the repo root, `classicmodels` loaded, `.env` configured. Use `API_BASE_URL` if the API is not at `http://127.0.0.1:8000`.
- **Task 1:** Direct `MySQLDataService` call.
- **Tasks 2–4:** Direct `CustomerResource`, `OrderResource`, `OrderDetailsResource`.
- **Task 5:** Every required route: **GET/POST** on `/customers`, `/orders`, `/orderdetails`; **GET/PUT/DELETE** on `/customers/{customerNumber}`, `/orders/{orderNumber}`, `/orders/{orderNumber}/orderdetails/{productCode}` (composite key as implemented in `main.py`).

A closing section **creates scratch rows** (customer → order → order line), exercises **PUT** on each, then **DELETEs** in dependency order (detail → order → customer). Use a disposable DB or accept small transient inserts if you interrupt before cleanup.

```
[15]: import os
import time

import requests
from urllib.parse import quote

BASE_URL = os.environ.get("API_BASE_URL", "http://127.0.0.1:8000").rstrip("/")

try:
    _health = requests.get(f"{BASE_URL}/health", timeout=5)
    _health.raise_for_status()
except requests.RequestException as exc:
    raise RuntimeError(
        f"Cannot reach API at {BASE_URL!r}. Start the server from the repo_
↳root, then rerun this cell."
    ) from exc

print("OK:", BASE_URL)
```

OK: http://127.0.0.1:8000

1.1 Sample rows from classicmodels (used below for reads and FK-safe writes)

```
[16]: r = requests.get(f"{BASE_URL}/customers", timeout=30)
assert r.status_code == 200, r.text
cust_items = r.json()["items"]
assert len(cust_items) > 0
SAMPLE_CUSTOMER_NUMBER = cust_items[0]["customerNumber"]

r = requests.get(f"{BASE_URL}/orders", timeout=30)
assert r.status_code == 200, r.text
order_items = r.json()["items"]
assert len(order_items) > 0
SAMPLE_ORDER_NUMBER = order_items[0]["orderNumber"]

r = requests.get(
    f"{BASE_URL}/orderdetails",
    params={"orderNumber": SAMPLE_ORDER_NUMBER},
    timeout=30,
)
assert r.status_code == 200, r.text
od_lines = r.json()["items"]
assert len(od_lines) > 0
SAMPLE_PRODUCT_CODE = od_lines[0]["productCode"]

print("SAMPLE_CUSTOMER_NUMBER", SAMPLE_CUSTOMER_NUMBER)
print("SAMPLE_ORDER_NUMBER", SAMPLE_ORDER_NUMBER)
print("SAMPLE_PRODUCT_CODE", SAMPLE_PRODUCT_CODE)
```

```
SAMPLE_CUSTOMER_NUMBER 103
SAMPLE_ORDER_NUMBER 10100
SAMPLE_PRODUCT_CODE S18_1749
```

1.2 Task 1 — MySQLDataService (direct)

Same connection settings as the app (load_mysql_env / .env).

```
[17]: import sys
from pathlib import Path

try:
    from dotenv import load_dotenv
except ImportError:
    load_dotenv = None

_cwd = Path.cwd().resolve()
PROJECT_ROOT = _cwd if (_cwd / "app" / "main.py").exists() else _cwd.parent
```

```

sys.path.insert(0, str(PROJECT_ROOT))
if load_dotenv:
    load_dotenv(PROJECT_ROOT / ".env")

from app.services.MySQLDataService import MySQLDataService, load_mysql_env

_db = load_mysql_env()
svc = MySQLDataService(
    {
        **_db,
        "table": "customers",
        "primary_key_columns": ["customerNumber"],
        "integer_columns": ["customerNumber", "salesRepEmployeeNumber"],
        "float_columns": ["creditLimit"],
        "auto_increment_pk": True,
    }
)
row = svc.retrieveByPrimaryKey(str(SAMPLE_CUSTOMER_NUMBER))
assert row
row["customerNumber"], row["customerName"]

```

[17]: (103, 'Atelier graphique')

1.3 Tasks 2–4 — Resources (CustomerResource, OrderResource, OrderDetailsResource)

```

[18]: from app.resources.CustomerResource import CustomerResource
from app.resources.OrderResource import OrderResource
from app.resources.OrderDetailsResource import OrderDetailsResource

cr, or_, odr = CustomerResource(), OrderResource(), OrderDetailsResource()
len(cr.get({}).items), len(or_.get({}).items), len(odr.get({}).items)

```

[18]: (122, 326, 2996)

1.4 Task 5 — Customers (HTTP)

Collection: GET, POST /customers. Single: GET, PUT, DELETE /customers/{customerNumber}.

```

[19]: # GET collection (+ equality template per README / Harry Potter pattern)
r = requests.get(f"{BASE_URL}/customers", timeout=30)
assert r.status_code == 200
r = requests.get(f"{BASE_URL}/customers", params={"country": "France"},
    ↪ timeout=30)
assert r.status_code == 200
len(r.json()["items"])

```

[19]: 12

```
[20]: # GET one / missing → 404
r = requests.get(f"{BASE_URL}/customers/{SAMPLE_CUSTOMER_NUMBER}", timeout=30)
assert r.status_code == 200
r.json()

missing = requests.get(f"{BASE_URL}/customers/99999999", timeout=30)
assert missing.status_code == 404
missing.json()
```

[20]: {'detail': "No customer with customerNumber '99999999'"}

1.5 Task 5 — Orders (HTTP)

Collection: GET, POST /orders. **Single:** GET, PUT, DELETE /orders/{orderNumber}.

```
[21]: r = requests.get(f"{BASE_URL}/orders", timeout=30)
assert r.status_code == 200
r = requests.get(f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}", timeout=30)
assert r.status_code == 200
r.json()
```

[21]: {'orderNumber': 10100,
 'orderDate': '2003-01-06',
 'requiredDate': '2003-01-13',
 'shippedDate': '2003-01-10',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 363}

```
[22]: missing = requests.get(f"{BASE_URL}/orders/99999998", timeout=30)
assert missing.status_code == 404
missing.json()
```

[22]: {'detail': "No order with orderNumber '99999998'"}

1.6 Task 5 — Order details (HTTP)

Collection: GET, POST /orderdetails. **Single:** GET, PUT, DELETE on /orders/{orderNumber}/orderdetails/{productCode} (composite PK).

```
[23]: r = requests.get(f"{BASE_URL}/orderdetails", timeout=30)
assert r.status_code == 200
r = requests.get(
    f"{BASE_URL}/orderdetails",
    params={"orderNumber": SAMPLE_ORDER_NUMBER},
    timeout=30,
```

```

)
assert r.status_code == 200
r = requests.get(f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails",
    ↪timeout=30)
assert r.status_code == 200
len(r.json()["items"])

```

[23]: 4

```

[24]: pc_q = quote(SAMPLE_PRODUCT_CODE, safe='')
r = requests.get(
    f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails/{pc_q}",
    timeout=30,
)
assert r.status_code == 200
r.json()

```

```

[24]: {'orderNumber': 10100,
      'productCode': 'S18_1749',
      'quantityOrdered': 30,
      'priceEach': 136.0,
      'orderLineNumber': 3}

```

```

[25]: missing = requests.get(
    f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails/"
    ↪f"{quote('___NO_SUCH_PRODUCT___', safe='')}",
    timeout=30,
)
assert missing.status_code == 404
missing.json()

```

```

[25]: {'detail': "No order detail with key '10100|___NO_SUCH_PRODUCT___'"}

```

1.7 Task 5 — End-to-end POST → PUT → DELETE (scratch customer → order → line)

Creates rows, updates them, then deletes (**order detail first**, then **order**, then **customer**) so FK constraints stay valid.

```

[26]: def _parse_new_pk(resp: requests.Response) -> int:
    resp.raise_for_status()
    j = resp.json()
    if isinstance(j, str) and j.isdigit():
        return int(j)
    if isinstance(j, int):
        return j
    return int(str(j).strip('"'))

```

```

suffix = str(int(time.time()))
scratch_name = f"Notebook Scratch {suffix}"

# POST /customers
post_c = requests.post(
    f"{BASE_URL}/customers",
    json={
        "customerName": scratch_name,
        "contactLastName": "Test",
        "contactFirstName": "Nb",
        "phone": "555-0100",
        "addressLine1": "1 Scratch Lane",
        "city": "NYC",
        "country": "USA",
    },
    timeout=30,
)
SCRATCH_CUSTOMER_NUMBER = _parse_new_pk(post_c)
SCRATCH_CUSTOMER_NUMBER

```

[26]: 497

```

[27]: # POST /orders (FK: customerNumber)
post_o = requests.post(
    f"{BASE_URL}/orders",
    json={
        "orderDate": "2026-05-03",
        "requiredDate": "2026-05-10",
        "shippedDate": None,
        "status": "In Process",
        "comments": "task6 notebook",
        "customerNumber": SCRATCH_CUSTOMER_NUMBER,
    },
    timeout=30,
)
SCRATCH_ORDER_NUMBER = _parse_new_pk(post_o)
SCRATCH_ORDER_NUMBER

```

[27]: 10426

```

[28]: # POST /orderdetails (FK: orderNumber, productCode)
post_od = requests.post(
    f"{BASE_URL}/orderdetails",
    json={
        "orderNumber": SCRATCH_ORDER_NUMBER,
        "productCode": SAMPLE_PRODUCT_CODE,
        "quantityOrdered": 2,
    },
)

```

```

        "priceEach": 99.99,
        "orderLineNumber": 1,
    },
    timeout=30,
)
assert post_od.status_code == 200, post_od.text
post_od.text.strip()

```

[28]: '"10426|S18_1749"'

```

[29]: # GET scratch resources
requests.get(f"{BASE_URL}/customers/{SCRATCH_CUSTOMER_NUMBER}", timeout=30).
    ↪ json()

requests.get(f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}", timeout=30).json()

requests.get(
    f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}/orderdetails/"
    ↪ f"{quote(SAMPLE_PRODUCT_CODE, safe='')}",
    timeout=30,
).json()

```

[29]: {'orderNumber': 10426,
'productCode': 'S18_1749',
'quantityOrdered': 2,
'priceEach': 99.99,
'orderLineNumber': 1}

```

[30]: # PUT /customers/{customerNumber}
c = requests.get(f"{BASE_URL}/customers/{SCRATCH_CUSTOMER_NUMBER}", timeout=30).
    ↪ json()
c["customerName"] = scratch_name + " Updated"
r = requests.put(f"{BASE_URL}/customers/{SCRATCH_CUSTOMER_NUMBER}", json=c,
    ↪ timeout=30)
assert r.status_code == 200 and r.json() == {"updated": 1}

# PUT /orders/{orderNumber}
o = requests.get(f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}", timeout=30).json()
o["status"] = "Shipped"
o["comments"] = "updated"
r = requests.put(f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}", json=o,
    ↪ timeout=30)
assert r.status_code == 200 and r.json() == {"updated": 1}

# PUT /orders/.../orderdetails/{productCode}
od = requests.get(

```

```

    f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}/orderdetails/
↪{quote(SAMPLE_PRODUCT_CODE, safe='')}",
    timeout=30,
).json()
od["quantityOrdered"] = 5
r = requests.put(
    f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}/orderdetails/
↪{quote(SAMPLE_PRODUCT_CODE, safe='')}",
    json=od,
    timeout=30,
)
assert r.status_code == 200 and r.json() == {"updated": 1}

"updated OK"

```

[30]: 'updated OK'

```

[31]: # DELETE order line + order + customer
r = requests.delete(
    f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}/orderdetails/
↪{quote(SAMPLE_PRODUCT_CODE, safe='')}",
    timeout=30,
)
assert r.status_code == 200 and r.json()["deleted"] == 1

r = requests.delete(f"{BASE_URL}/orders/{SCRATCH_ORDER_NUMBER}", timeout=30)
assert r.status_code == 200 and r.json()["deleted"] == 1

r = requests.delete(f"{BASE_URL}/customers/{SCRATCH_CUSTOMER_NUMBER}",
↪timeout=30)
assert r.status_code == 200 and r.json()["deleted"] == 1

gone = requests.get(f"{BASE_URL}/customers/{SCRATCH_CUSTOMER_NUMBER}",
↪timeout=30)
assert gone.status_code == 404

"cleanup OK"

```

[31]: 'cleanup OK'

[]: